

Parser de Comandos de RPG de Mesa (Web)

Relatório técnico - Linguagens Formais e Autômatos

Integrantes: preencher os nomes completos em docs/contribuicoes.md. Repositório configurado: <https://github.com/RafaelFerreira18/parser-comandos-rpg-mesa-web>. A implementação está local e a publicação ainda precisa ser concluída.

1. Problema e solução

Comandos de RPG digitados em texto exigem reconhecer a ação, validar os parâmetros e calcular resultados. O projeto oferece interface web com atualização sem recarregar a página e menu CLI. Cinco expressões regulares relevantes reconhecem rolagem, ataque, magia, cura e teste de atributo.

Entrada: texto do comando. Processamento: `re.fullmatch`, captura nomeada, conversão de valores e regras de rolagem. Saída: mensagem clara, parâmetros, valores individuais, total, sucesso/falha, dano ou cura. A interface distingue erros de sintaxe de falhas da ação no jogo.

2. Implementação e arquitetura

Python 3.10+ (recomendado 3.11+) com `re`, `dataclasses`, `random`, `http.server`, `json`, `argparse` e `unittest`. JavaScript usa `fetch` para consultar o backend. A aplicação não depende de frameworks nem APIs externas. Graphviz e JFLAP são ferramentas de documentação dos autômatos.

Módulos: `language.py` define as ER; `engine.py` valida e executa; `automata.py` constrói e simula AFNe; `server.py` expõe HTTP; `cli.py` apresenta o menu. `web/` contém HTML, CSS e JavaScript. `tools/` gera as fichas, diagramas e PDFs. Testes usam a mesma API pública que as interfaces.

3. Contrato de entrada e regras de jogo

Espaços U+0020 e minúsculas são obrigatórios. A validação não aplica `strip`, flags de Unicode abreviadas ou conversão de caixa. Quantidade 1-20; faces 4, 6, 8, 10, 12, 20, 100; modificador opcional -99 a +99; CA/CD 1-99 sem zero inicial; bônus com sinal opcional e um ou dois dígitos. Nomes seguem `[a-z][a-z0-9_]*`. Bônus e modificadores permitem zeros iniciais com até dois dígitos.

Ataque: $d20 + \text{bônus} \geq \text{CA}$ causa o dano rolado; falha causa zero. Teste: $d20 + \text{bônus} \geq \text{CD}$ produz sucesso. Magia: o alvo faz resistência $d20 + \text{bônus}$ contra CD; se resistir, recebe metade do dano, arredondada para baixo. Sucesso da magia significa falha da resistência. Cura e dano têm piso zero, mas o total de uma rolagem simples pode ser negativo. Não há críticos ou fichas persistentes.

Limites operacionais: comandos até 512 caracteres, corpo HTTP até 4096 bytes e leitura com timeout. A regex e o AFNe admitem nomes de comprimento arbitrário. O serviço aplica seus limites separadamente, sem redefinir as cinco linguagens regulares.

4. Notação formal e equivalência

As macros abaixo são abreviações de expressões regulares formais. Literais de várias letras significam concatenação de cada símbolo. O alfabeto de cada ficha lista somente os símbolos efetivamente usados. A ficha

Markdown inclui a expansão completa, sem macros.

$D = (0 \cup 1 \cup 2 \cup 3 \cup 4 \cup 5 \cup 6 \cup 7 \cup 8 \cup 9)$

$U = (1 \cup 2 \cup 3 \cup 4 \cup 5 \cup 6 \cup 7 \cup 8 \cup 9)$

$L = (a \cup b \cup c \cup \dots \cup z)$, somente as 26 letras ASCII

$I = L (L \cup D \cup _)^*$; $N = U (\epsilon \cup D)$

$B = (\epsilon \cup '+' \cup '-') D (\epsilon \cup D)$

$Q = (U \cup '1' D \cup '20')$

$F = ('4' \cup '6' \cup '8' \cup '10' \cup '12' \cup '20' \cup '100')$

$R = Q 'd' F (\epsilon \cup ('+' \cup '-') D (\epsilon \cup D))$

$A = ('forca' \cup 'destreza' \cup 'constituicao' \cup 'inteligencia' \cup 'sabedoria' \cup 'carisma')$

SP = espaço U+0020. Aspas delimitam literais, não pertencem à entrada.

União $\cup$ gera alternância (?...|...). Justaposição gera concatenação. Fecho $*$ gera zero ou mais repetições.

Opcionalidade corresponde a união com ϵ . Classes finitas expandem união de símbolos. Grupos (?P<nome>...) só capturam parâmetros. re.fullmatch exige a cadeia inteira, sem incluir âncoras no alfabeto. Não se usam retroreferências, lookahead, condicionais ou recursão.

Construção do AFN ϵ : um literal usa uma transição por símbolo; uma classe usa transições paralelas; concatenação compartilha fronteiras; união acrescenta ramificações ϵ ; fecho acrescenta passagem vazia e retorno ϵ . Por indução sobre a árvore de operadores, cada transformação preserva a linguagem. A simulação fecha movimentos ϵ antes e depois de cada símbolo.

ER-01 - Rolagem de dados

Finalidade: Extrair quantidade, faces e modificador de uma rolagem.

Linguagem: comandos completos conforme o exemplo /rolar 2d6+3, com domínios e ordem dos campos definidos na seção 3.

$\Sigma = \{\text{SP}, +, -, /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \text{a}, \text{d}, \text{l}, \text{o}, \text{r}\}$

ER formal (macros da seção 4): '/rolar' SP R

Padrão exato gerado e utilizado por Rule.pattern:

```
/rolar\ ( ?P<dados>(?:[123456789]|1[0123456789]|20)d(?:4|6|8|10|12|20|100)(?:|[\+\-\ ][0123456789](?:|[0123456789]))))
```

A quebra de linha visual do padrão não insere caracteres na expressão.

AFNε: início q0, final q1, 49 estados, 94 transições elementares. Diagrama integral no apêndice e em automata/rolar.svg. XML JFLAP e tabela de transições acompanham o repositório.

Operadores: união nas alternativas e opcionais; concatenação dos campos; fecho no nome quando há identificador. A captura nomeada extrai valores sem alterar aceitação.

Seis cadeias aceitas:

```
/rolar 1d4  
/rolar 20d100+99  
/rolar 2d6-3  
/rolar 10d10+0  
/rolar 1d20-99  
/rolar 3d12+05
```

Seis cadeias rejeitadas:

```
ε (entrada vazia)  
'/rolar 0d6'  
'/rolar 21d6'  
'/rolar 1d7'  
'/rolar 1d6+100'  
'/rolar 1d6\n'
```

Casos-limite: mínimos na primeira aceita, máximos entre as demais, vazio e valores fora do domínio nas rejeitadas. Todas produziram o resultado esperado nas regex, no simulador AFNε e nos XML exportados.

ER-02 - Ataque

Finalidade: Extrair alvo, bônus de acerto, classe de armadura e dano.

Linguagem: comandos completos conforme o exemplo /atacar goblin bonus=5 ca=14 dano=1d8+2, com domínios e ordem dos campos definidos na seção 3.

$\Sigma = \{SP, +, -, /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, =, _, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$

ER formal (macros da seção 4): '/atacar' SP I SP 'bonus=' B SP 'ca=' N SP 'dano=' R

Padrão exato gerado e utilizado por Rule.pattern:

```
/atacar\ (?P<alvo>[abcdefghijklmnopqrstuvwxyz](?:[abcdefghijklmnopqrstuvwxyz0123456789_])*)\
bonus=(?P<bonus>(?:[\+\- ])[0123456789](?:[0123456789]))\
ca=(?P<ca>[123456789](?:[0123456789]))\ dano=(?P<dados>(?:[123456789]|1[0123456789]|20)d(?:4
|6|8|10|12|20|100)(?:[\+\- ])[0123456789](?:[0123456789]))
```

A quebra de linha visual do padrão não insere caracteres na expressão.

AFNε: início q0, final q1, 88 estados, 235 transições elementares. Diagrama integral no apêndice e em automata/atacar.svg. XML JFLAP e tabela de transições acompanham o repositório.

Operadores: união nas alternativas e opcionais; concatenação dos campos; fecho no nome quando há identificador. A captura nomeada extrai valores sem alterar aceitação.

Seis cadeias aceitas:

```
/atacar a bonus=0 ca=1 dano=1d4
/atacar goblin bonus=5 ca=14 dano=1d8+2
/atacar orc_2 bonus=-99 ca=99 dano=20d100+99
/atacar dragao bonus=+9 ca=20 dano=2d12
/atacar x1 bonus=00 ca=2 dano=1d6-99
/atacar lobo bonus=-1 ca=12 dano=3d10+05
```

Seis cadeias rejeitadas:

```
ε (entrada vazia)
'/atacar 1orc bonus=0 ca=1 dano=1d4'
'/atacar orc bonus=100 ca=10 dano=1d6'
'/atacar orc bonus=0 ca=0 dano=1d6'
'/atacar orc ca=10 bonus=0 dano=1d6'
'/atacar orc bonus=0 ca=10 dano=0d6'
```

Casos-limite: mínimos na primeira aceita, máximos entre as demais, vazio e valores fora do domínio nas rejeitadas. Todas produziram o resultado esperado nas regex, no simulador AFNε e nos XML exportados.

ER-03 - Magia ofensiva

Finalidade: Extrair magia, alvo, dificuldade, bônus de resistência e dano.

Linguagem: comandos completos conforme o exemplo /magia fogo alvo=goblin cd=12 bonus=3 dano=2d6, com domínios e ordem dos campos definidos na seção 3.

$\Sigma = \{SP, +, -, /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, =, _, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$

ER formal (macros da seção 4): '/magia' SP I SP 'alvo=' I SP 'cd=' N SP 'bonus=' B SP 'dano=' R

Padrão exato gerado e utilizado por Rule.pattern:

```
/magia\ (?P<magia>[abcdefghijklmnopqrstuvwxyz](?:[abcdefghijklmnopqrstuvwxyz0123456789_])*)\
alvo=(?P<alvo>[abcdefghijklmnopqrstuvwxyz](?:[abcdefghijklmnopqrstuvwxyz0123456789_])*)\
cd=(?P<cd>[123456789](?:[0123456789]))\
bonus=(?P<bonus>(?:[\+\-\_])[0123456789](?:[0123456789]))\ dano=(?P<dados>(?:[123456789]|1[01
23456789]|20)d(?:4|6|8|10|12|20|100)(?:[\+\-\_][0123456789](?:[0123456789]))))
```

A quebra de linha visual do padrão não insere caracteres na expressão.

AFNε: início q0, final q1, 97 estados, 307 transições elementares. Diagrama integral no apêndice e em automata/magia.svg. XML JFLAP e tabela de transições acompanham o repositório.

Operadores: união nas alternativas e opcionais; concatenação dos campos; fecho no nome quando há identificador. A captura nomeada extrai valores sem alterar aceitação.

Seis cadeias aceitas:

```
/magia a alvo=b cd=1 bonus=0 dano=1d4
/magia fogo alvo=goblin cd=12 bonus=3 dano=2d6
/magia gelo_2 alvo=orc_1 cd=99 bonus=-99 dano=20d100+99
/magia raio alvo=a1 cd=20 bonus=+5 dano=1d20-99
/magia fogo alvo=ana cd=2 bonus=00 dano=3d8
/magia luz alvo=elfo cd=15 bonus=-1 dano=2d12+05
```

Seis cadeias rejeitadas:

```
ε (entrada vazia)
'/magia 2fogo alvo=b cd=1 bonus=0 dano=1d4'
'/magia fogo alvo=_a cd=1 bonus=0 dano=1d4'
'/magia fogo alvo=b cd=00 bonus=0 dano=1d4'
'/magia fogo alvo=b cd=1 bonus=0 dano=1d7'
'/magia fogo alvo=b bonus=0 cd=1 dano=1d4'
```

Casos-limite: mínimos na primeira aceita, máximos entre as demais, vazio e valores fora do domínio nas rejeitadas. Todas produziram o resultado esperado nas regex, no simulador AFNε e nos XML exportados.

ER-04 - Cura

Finalidade: Extrair alvo e dados dos pontos de vida recuperados.

Linguagem: comandos completos conforme o exemplo /curar ana 2d8+3, com domínios e ordem dos campos definidos na seção 3.

$\Sigma = \{SP, +, -, /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, _, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$

ER formal (macros da seção 4): '/curar' SP I SP R

Padrão exato gerado e utilizado por Rule.pattern:

```
/curar\ (?P<alvo>[abcdefghijklmnopqrstuvwxyz](?:[abcdefghijklmnopqrstuvwxyz0123456789_])*)\ (
?P<dados>(?:[123456789]|1[0123456789]|20)d(?:4|6|8|10|12|20|100)(?:|[\+|-
][0123456789](?:|[0123456789]))))
```

A quebra de linha visual do padrão não insere caracteres na expressão.

AFNε: início q0, final q1, 54 estados, 162 transições elementares. Diagrama integral no apêndice e em automata/curar.svg. XML JFLAP e tabela de transições acompanham o repositório.

Operadores: união nas alternativas e opcionais; concatenação dos campos; fecho no nome quando há identificador. A captura nomeada extrai valores sem alterar aceitação.

Seis cadeias aceitas:

```
/curar a 1d4
/curar ana 2d8+3
/curar elfo_2 20d100+99
/curar x1 1d20-99
/curar paladino 3d12+05
/curar clerigo 10d10+0
```

Seis cadeias rejeitadas:

```
ε (entrada vazia)
'/curar 1ana 1d4'
'/curar Ana 1d4'
'/curar ana 21d6'
'/curar ana 1d6+100'
'/curar ana 1d4'
```

Casos-limite: mínimos na primeira aceita, máximos entre as demais, vazio e valores fora do domínio nas rejeitadas. Todas produziram o resultado esperado nas regex, no simulador AFNε e nos XML exportados.

ER-05 - Teste de atributo

Finalidade: Extrair atributo, bônus e classe de dificuldade.

Linguagem: comandos completos conforme o exemplo /teste forca bonus=-2 cd=15, com domínios e ordem dos campos definidos na seção 3.

$\Sigma = \{SP, +, -, /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, =, a, b, c, d, e, f, g, i, l, m, n, o, r, s, t, u, z\}$

ER formal (macros da seção 4): '/teste' SP A SP 'bonus=' B SP 'cd=' N

Padrão exato gerado e utilizado por Rule.pattern:

```
/teste\ (?P<atributo>(?:forca|destreza|constituicao|inteligencia|sabedoria|carisma))\  
bonus=(?P<bonus>(?:|[\+\-])?[0123456789])(?:|[0123456789]))\  
cd=(?P<cd>[123456789])(?:|[0123456789]))
```

A quebra de linha visual do padrão não insere caracteres na expressão.

AFNε: início q0, final q1, 96 estados, 139 transições elementares. Diagrama integral no apêndice e em automata/teste.svg. XML JFLAP e tabela de transições acompanham o repositório.

Operadores: união nas alternativas e opcionais; concatenação dos campos; fecho no nome quando há identificador. A captura nomeada extrai valores sem alterar aceitação.

Seis cadeias aceitas:

```
/teste forca bonus=0 cd=1  
/teste destreza bonus=-99 cd=99  
/teste constituicao bonus=+5 cd=12  
/teste inteligencia bonus=00 cd=20  
/teste sabedoria bonus=-1 cd=15  
/teste carisma bonus=99 cd=2
```

Seis cadeias rejeitadas:

```
ε (entrada vazia)  
'/teste sorte bonus=0 cd=1'  
'/teste forca bonus=100 cd=1'  
'/teste forca bonus=0 cd=0'  
'/teste forca cd=1 bonus=0'  
'/teste forca bonus=0 cd=1 '
```

Casos-limite: mínimos na primeira aceita, máximos entre as demais, vazio e valores fora do domínio nas rejeitadas. Todas produziram o resultado esperado nas regex, no simulador AFNε e nos XML exportados.

5. Testes e análise dos resultados

A execução registrada em docs/resultados-testes.txt concluiu 14 métodos de teste com OK. A suíte verifica 60 cadeias obrigatórias, cada uma nas regex e no AFNε, e repete os casos nos .jff exportados. Mais 1.500 mutações com semente 42 confrontam a regex e a simulação independente. Nenhuma divergência foi encontrada nessa amostra.

Os testes determinísticos verificam igualdade no limiar, ataque falho sem rolar dano, resistência à magia com metade do dano, piso zero, totais negativos, todos os dados e quantidade máxima. Testes HTTP reais verificam arquivos estáticos, validação sem rolagem, execução, JSON malformado, tipo incorreto, corpo grande e rota inexistente. Testes por amostragem não provam equivalência exaustiva.

A leitura e simulação dos arquivos JFLAP confirmaram estados, início, final, transições e cadeias. A interface gráfica JFLAP 7 não foi executada neste ambiente; a equipe deve conferir abertura e Multiple Run na máquina de apresentação.

6. Execução e demonstração

```
python -m rpg.server
python -m rpg.cli
python -m unittest discover -v
```

Acesse <http://127.0.0.1:8000>. Digite /rolar 2d6+3 e execute. Depois mostre a rejeição de /rolar 21d6. A digitação consulta /api/validar após 300 ms; /api/executar lança os dados ao enviar. O menu CLI possui executar, exemplos e sair. Ctrl+C encerra a aplicação. A apresentação usa o roteiro de 10-12 minutos do repositório.

7. Limitações e melhorias

Este sistema não reproduz um regulamento oficial. Não há críticos, vantagem, cadastro de personagens, persistência de PV, catálogo de magias ou autenticação. A aleatoriedade de produção usa SystemRandom; testes injetam resultados controlados. O servidor local é destinado à demonstração. Melhorias possíveis: fichas, histórico persistente e regras adicionais, com novas ER e autômatos equivalentes.

8. Contribuições, IA e entrega

Identificação e tarefas reais de 1 a 4 integrantes devem ser preenchidas em docs/contribuicoes.md antes do envio. OpenAI Codex apoiou implementação inicial, documentação, autômatos, testes e apresentação. A equipe deve compreender e revisar o conteúdo, assumir autoria e conseguir alterá-lo. O endereço do repositório provém da configuração Git existente; a implementação desta entrega permanece local até a publicação.

A entrega exige link definitivo GitHub, relatório, apresentação e nomes no Classroom. Os documentos divergem quanto ao prazo: lauda traz 02/10/2026 no cabeçalho e 30/09/2026 em uma etapa; guia traz 30/09/2026. Confirmar o prazo com o professor.

9. Referências

Lauda do trabalho de Linguagens Formais e Autômatos fornecida pela equipe.

GUIA_SINTAXE_EXPRESSOES_REGULARES_TRABALHO_LFA - FINAL.pdf, material do professor.

Python: <https://docs.python.org/3/library/re.html>; [unittest.html](https://docs.python.org/3/library/unittest.html); [http.server.html](https://docs.python.org/3/library/http.server.html); [random.html](https://docs.python.org/3/library/random.html).

Graphviz: <https://graphviz.org/>. SVGs gerados pelo Graphviz via @viz-js/viz.

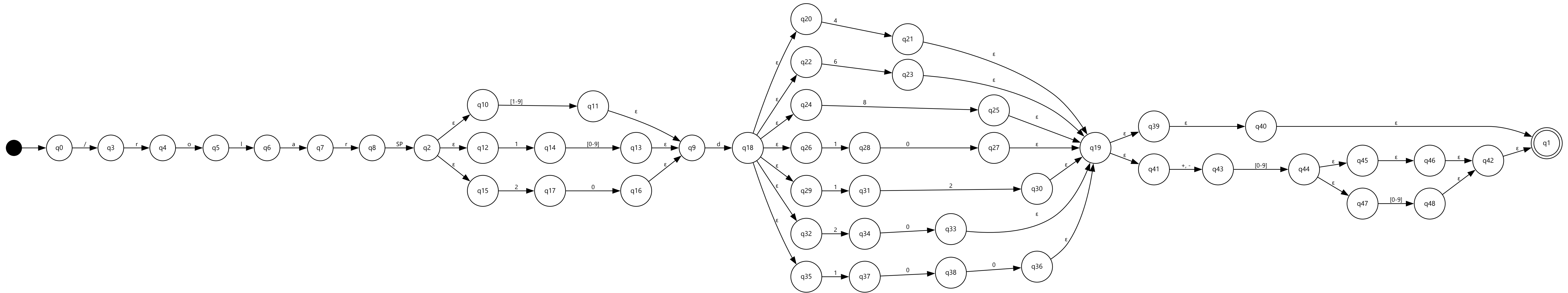
JFLAP: <https://www.jflap.org/>. Formato XML de autômatos finitos.

Apêndice: cinco AFNε completos

As páginas seguintes usam tamanho ampliado para manter estados e rótulos legíveis com zoom. SVGs são a opção indicada para projeção por trechos. As tabelas completas estão em docs/expressoes-regulares.md.

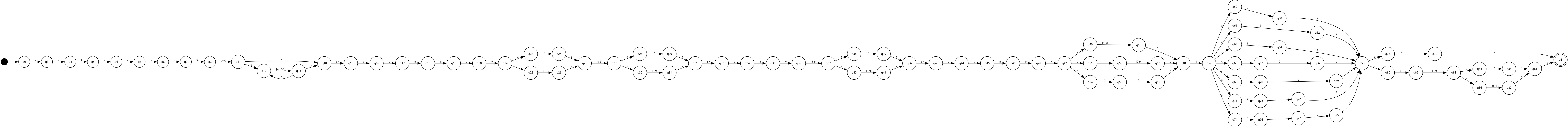
ER-01 - Rolagem de dados - AFNε completo

Página de grande formato para zoom. Inicial q0, final q1. Classes = transições paralelas. SP = espaço. ε = movimento vazio.



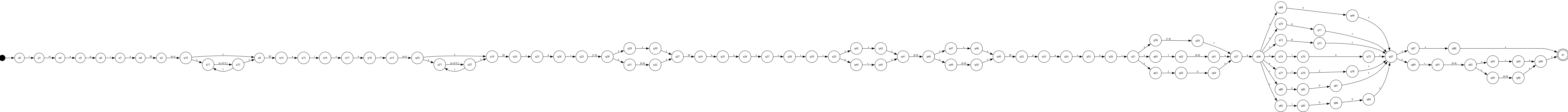
IR-02 - Ataque - AFNε completo

ágina de grande formato para zoom. Inicial q0, final q1. Classes = transições paralelas. SP = espaço. ϵ = movimento vazio.



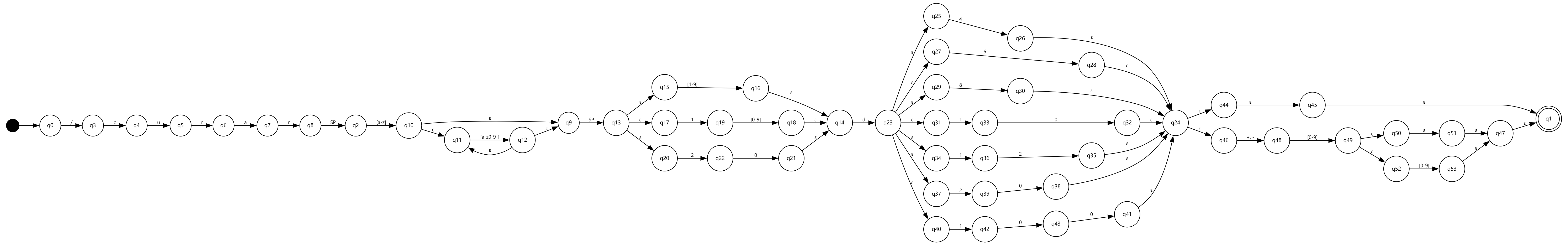
R-03 - Magia ofensiva - AFNε completo

gina de grande formato para zoom. Inicial q0, final q1. Classes = transições paralelas. SP = espaço. ϵ = movimento vazio.



ER-04 - Cura - AFNε completo

Página de grande formato para zoom. Inicial q0, final q1. Classes = transições paralelas. SP = espaço. ε = movimento vazio.



ER-05 - Teste de atributo - AFNε completo

Página de grande formato para zoom. Inicial q0, final q1. Classes = transições paralelas. SP = espaço. ε = movimento vazio.

